

04 — Deployment vs DeploymentConfig

Modulo: 3 — Deploy di Applicazioni
Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Comprendere la struttura e le funzionalità di **Deployment** (Kubernetes standard)
- Comprendere la struttura e le funzionalità di **DeploymentConfig** (OpenShift legacy)
- Distinguere le risorse correlate: **ReplicaSet** vs **ReplicationController**
- Eseguire la migrazione da **DeploymentConfig** a **Deployment** con procedura step-by-step
- Usare **oc rollout** per pause, resume, rollback e cronologia su entrambe le risorse
- Applicare la strategia di aggiornamento corretta (**RollingUpdate** vs **Recreate**)

1. Teoria e Concetti Chiave

Deployment — La Risorsa Kubernetes Standard

Il **Deployment** è la risorsa Kubernetes standard per gestire applicazioni stateless. Gestisce un insieme di Pod tramite un **ReplicaSet** che viene creato e aggiornato automaticamente.

Flusso di aggiornamento di un Deployment:

Deployment (spec.template aggiornato)

- └─ crea nuovo ReplicaSet v2 (scala da 0 a N)
- └─ scala vecchio ReplicaSet v1 da N a 0
- └─ vecchio ReplicaSet conservato (per rollback)

DeploymentConfig — La Risorsa OpenShift Legacy

Il **DeploymentConfig** è una risorsa proprietaria OpenShift (API group **apps.openshift.io**) introdotta nelle versioni precedenti di OCP quando **Deployment** Kubernetes non aveva ancora tutte le funzionalità necessarie. Gestisce i Pod tramite un **ReplicationController**.

 **Attenzione:** **DeploymentConfig** è **deprecato** a partire da OpenShift 4.14. Red Hat raccomanda l'uso di **Deployment** per tutti i nuovi progetti e la migrazione dei progetti esistenti.

Tabella Comparativa Completa

Caratteristica	Deployment	DeploymentConfig
API Group	apps/v1 (Kubernetes standard)	apps.openshift.io/v1 (OpenShift only)
Risorsa subordinata	ReplicaSet	ReplicationController

Caratteristica	Deployment	DeploymentConfig
Trigger ImageChange	✗ (gestito da ImageStream separato)	✓ Nativo (<code>.spec.triggers</code>)
Trigger ConfigChange	✗ (si usa GitOps/CI-CD)	✓ Nativo
Lifecycle Hooks	✗ (si usano init containers)	✓ <code>pre, mid, post</code>
Strategia Custom	✗	✓ Custom strategy
Rollback	<code>oc rollout undo</code>	<code>oc rollback</code> (deprecato)
Rolling update	✓ <code>RollingUpdate</code>	✓ <code>Rolling</code>
Recreate	✓ <code>Recreate</code>	✓ <code>Recreate</code>
Ecosistema	✓ Compatibile con tutti i tool K8s	⚠ Solo OCP
Stato	✓ Stabile, raccomandato	⚠ Deprecato in OCP 4.14+
Futuro	✓ Supporto a lungo termine	✗ Rimozione prevista

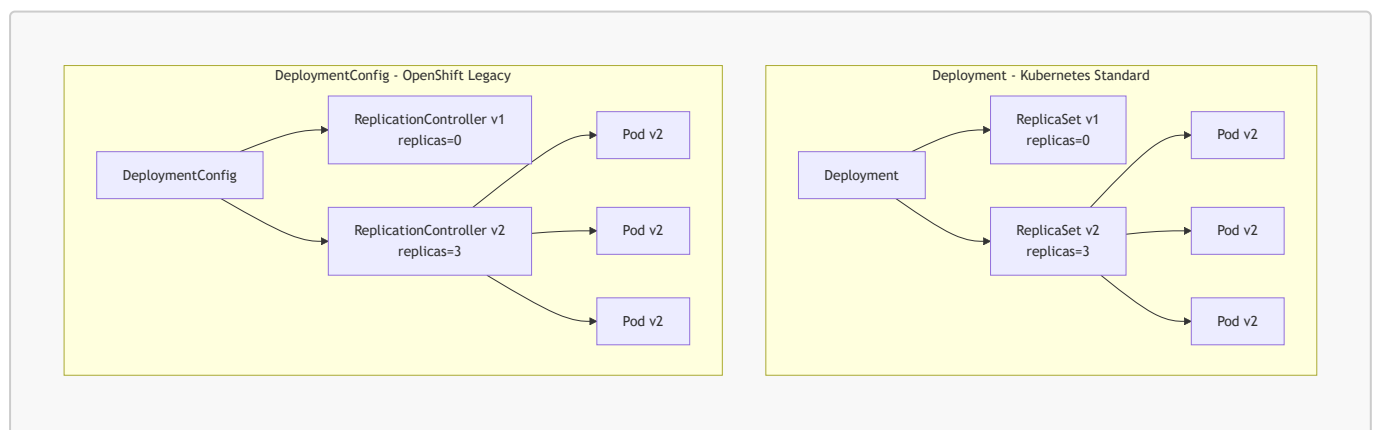
Strategie di Aggiornamento

RollingUpdate (Deployment): aggiornamento progressivo senza downtime. Ideale per applicazioni stateless.

- `maxSurge`: quanti Pod extra creare durante l'aggiornamento (es. `1` o `25%`)
- `maxUnavailable`: quanti Pod possono essere non disponibili durante l'aggiornamento (es. `0` o `25%`)

Recreate: tutti i Pod vecchi vengono eliminati prima di crearne di nuovi. Implica downtime. Usare per applicazioni che non supportano più versioni in parallelo (es. migrazioni DB, applicazioni con stato condiviso).

2. Diagramma — Confronto Architeturale



3. Comandi Pratici con Output Atteso

```
# Creare un Deployment (approccio imperativo)
oc create deployment my-app \
```

```

--image=registry.access.redhat.com/ubi9/ubi-minimal:latest \
--replicas=3 \
-n my-project

# Verificare il Deployment e i ReplicaSet creati
oc get deployment,replicaset -n my-project

# Output atteso:
# NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
# deployment.apps/my-app              3/3      3              3             30s
#
# NAME                                DESIRED    CURRENT    READY    AGE
# replicaset.apps/my-app-6d8b9c7f5    3          3          3        30s

# Verificare un DeploymentConfig esistente (legacy)
oc get deploymentconfig -n my-project

# Output atteso (se presente):
# NAME            REVISION    DESIRED    CURRENT    TRIGGERED BY
# legacy-app      3           2          2          config,image(legacy-app:latest)

# --- Rollout ---

# Verificare lo stato del rollout
oc rollout status deployment/my-app -n my-project
# Output: deployment "my-app" successfully rolled out

# Visualizzare la cronologia dei rollout
oc rollout history deployment/my-app -n my-project

# Output atteso:
# REVISION    CHANGE-CAUSE
# 1           <none>
# 2           Updated image to v1.1
# 3           Updated image to v1.2

# Visualizzare i dettagli di una revisione specifica
oc rollout history deployment/my-app --revision=2 -n my-project

# Eseguire il rollback all'ultima revisione stabile
oc rollout undo deployment/my-app -n my-project
# Output: deployment.apps/my-app rolled back

# Eseguire il rollback a una revisione specifica
oc rollout undo deployment/my-app --to-revision=1 -n my-project

# Sospendere un Deployment (blocca i trigger di rollout)
oc rollout pause deployment/my-app -n my-project
# Output: deployment.apps/my-app paused

# Riprendere il Deployment
oc rollout resume deployment/my-app -n my-project
# Output: deployment.apps/my-app resumed

```

```
# Forzare un nuovo rollout senza modifiche all'immagine (utile per ricaricare
ConfigMap/Secret)
oc rollout restart deployment/my-app -n my-project

# --- Confronto con DeploymentConfig ---

# Avviare manualmente un nuovo deploy su DeploymentConfig (legacy)
oc rollout latest deploymentconfig/legacy-app -n my-project

# Visualizzare la cronologia di un DeploymentConfig
oc rollout history deploymentconfig/legacy-app -n my-project

# Rollback su DeploymentConfig
oc rollout undo deploymentconfig/legacy-app -n my-project
```

4. Esempi YAML Completi

Deployment Kubernetes (raccomandato)

```
# file: deployment-my-app.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: my-project
  labels:
    app: my-app
  annotations:
    kubernetes.io/change-cause: "Aggiornamento a versione 1.2.0" # visibile in
rollout history
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app # deve corrispondere ai label del template Pod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1 # crea 1 Pod extra durante l'aggiornamento (totale: 4
durante update)
      maxUnavailable: 0 # nessun Pod non disponibile (zero downtime)
      minReadySeconds: 10 # attende 10s prima di considerare un Pod "pronto"
      progressDeadlineSeconds: 600 # fallisce se il rollout non completa entro 10
min
  revisionHistoryLimit: 5 # conserva 5 ReplicaSet per il rollback
  template:
    metadata:
      labels:
        app: my-app
        version: "1.2.0"
```

```
spec:
  containers:
  - name: my-app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    imagePullPolicy: IfNotPresent
    ports:
    - containerPort: 8080
  resources:
    requests:
      cpu: "100m"
      memory: "128Mi"
    limits:
      cpu: "500m"
      memory: "256Mi"
  readinessProbe:
    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10
    failureThreshold: 3
  livenessProbe:
    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 15
    periodSeconds: 20
```

DeploymentConfig OpenShift (legacy — solo per riferimento)

```
# file: deploymentconfig-legacy-app.yaml
# ⚠️ DEPRECATO in OCP 4.14+ — usare Deployment per nuovi progetti
apiVersion: apps.openshift.io/v1
kind: DeploymentConfig
metadata:
  name: legacy-app
  namespace: my-project
  labels:
    app: legacy-app
spec:
  replicas: 2
  selector:
    app: legacy-app
  strategy:
    type: Rolling
    rollingParams:
      maxSurge: 25%
      maxUnavailable: 25%
  pre:                                # Lifecycle hook pre-deploy
    failurePolicy: Abort
    execNewPod:
```

```

        command: ["/bin/sh", "-c", "echo 'Pre-deploy hook'"]
        containerName: legacy-app
    post:                                # Lifecycle hook post-deploy
        failurePolicy: Ignore
        execNewPod:
            command: ["/bin/sh", "-c", "echo 'Post-deploy hook'"]
            containerName: legacy-app
    triggers:
    - type: ImageChange                  # trigger automatico al cambio immagine
      imageChangeParams:
        automatic: true
        containerNames:
        - legacy-app
      from:
        kind: ImageStreamTag
        name: "legacy-app:latest"
    - type: ConfigChange                # trigger automatico al cambio configurazione
    template:
      metadata:
        labels:
          app: legacy-app
      spec:
        containers:
        - name: legacy-app
          image: " "                    # popolato automaticamente dal trigger
ImageChange
  ports:
  - containerPort: 8080

```

Migrazione da DeploymentConfig a Deployment

```

# Step 1: Esportare il DeploymentConfig esistente
oc get deploymentconfig legacy-app -n my-project -o yaml > legacy-app-dc.yaml

# Step 2: Il DeploymentConfig esportato va convertito manualmente
# Principali modifiche necessarie:
# - apiVersion: apps.openshift.io/v1 → apps/v1
# - kind: DeploymentConfig → Deployment
# - Rimuovere .spec.triggers (usare CI/CD o ImageStream annotation invece)
# - Rimuovere .spec.strategy.rollingParams → usare .spec.strategy.rollingUpdate
# - Cambiare .spec.selector da map a matchLabels
# - Sostituire lifecycle hooks con init containers o job post-deploy

# Step 3: Applicare il nuovo Deployment
oc apply -f my-app-deployment.yaml -n my-project

# Step 4: Verificare che il nuovo Deployment funzioni
oc rollout status deployment/my-app -n my-project

# Step 5: Scalare a zero il vecchio DeploymentConfig (non eliminarlo subito)
oc scale deploymentconfig/legacy-app --replicas=0 -n my-project

```

```
# Step 6: Dopo verifica, eliminare il DeploymentConfig
oc delete deploymentconfig/legacy-app -n my-project
```

5. Note Specifiche per Azure ARO

💡 Suggerimento ARO: Su ARO, il trigger `ImageChange` del `DeploymentConfig` non è necessario se si usa Azure Container Registry (ACR) con Azure Event Grid per notificare le push. Con un `Deployment` standard si può configurare una pipeline Azure DevOps che aggiorna l'immagine e esegue `oc set image`.

```
# Su ARO con pipeline Azure DevOps – aggiornare l'immagine di un Deployment
# (sostituisce il trigger ImageChange del DeploymentConfig)

# Nel task della pipeline Azure DevOps:
oc set image deployment/my-app \
  my-app=myregistry.azurecr.io/myapp:${Build.BuildId} \
  -n my-project

# Annotare il deployment con la causa del cambiamento (visibile in rollout
history)
oc annotate deployment/my-app \
  kubernetes.io/change-cause="Pipeline build ${Build.BuildId}" \
  --overwrite \
  -n my-project
```

Aspetto	OCP On-Premise	Azure ARO
Trigger ImageChange	DeploymentConfig o ImageStream	Consigliato: pipeline CI/CD + <code>oc set image</code>
Lifecycle hooks	DeploymentConfig <code>pre/post</code>	Init containers + Job post-deploy
Registry trigger	ImageStream interno	ACR + Event Grid + pipeline
Raccomandazione	Migrare a Deployment	Usare solo Deployment per nuovi progetti

6. Riepilogo dei Punti Chiave

- **Deployment** è la risorsa Kubernetes standard, raccomandata per tutti i nuovi progetti OCP 4.14+
- **DeploymentConfig** è deprecato in OCP 4.14+: offriva trigger nativi e lifecycle hooks, ma è OCP-specific e non compatibile con tool K8s standard
- Sotto un `Deployment` c'è un `ReplicaSet`; sotto un `DeploymentConfig` c'è un `ReplicationController`
- `oc rollout undo` funziona su entrambi; per i `DeploymentConfig` è preferibile `oc rollout undo deploymentconfig/<nome>`

- La migrazione da `DeploymentConfig` a `Deployment` richiede: modifica del `apiVersion/kind`, rimozione dei trigger, adattamento della strategia rolling e sostituzione dei lifecycle hooks
 - L'annotazione `kubernetes.io/change-cause` popola il campo `CHANGE-CAUSE` visibile con `oc rollout history`
 - Su **ARO** i trigger automatici vanno sostituiti con pipeline Azure DevOps che eseguono `oc set image`
-

7. Domande di Verifica

1. Quale risorsa subordinata gestisce i Pod di un `Deployment` Kubernetes?

- A) `ReplicationController`
- B) `ReplicaSet` ✓
- C) `StatefulSet`
- D) `DaemonSet`

2. Quale funzionalità è presente in `DeploymentConfig` ma NON in `Deployment`?

- A) Rolling update
- B) Strategia Recreate
- C) Rollback
- D) **Trigger `ImageChange` nativo e lifecycle hooks `pre/post`** ✓

3. Quale comando esegue un rollback alla revisione numero 2 di un `Deployment`?

- A) `oc rollout history deployment/my-app --revision=2`
- B) `oc rollback deployment/my-app --to-version=2`
- C) **`oc rollout undo deployment/my-app --to-revision=2`** ✓
- D) `oc set revision deployment/my-app 2`

4. Perché Red Hat raccomanda di migrare da `DeploymentConfig` a `Deployment`?

- A) `Deployment` supporta più repliche
- B) `DeploymentConfig` non supporta la strategia Recreate
- C) **`DeploymentConfig` è deprecato in OCP 4.14+ e non compatibile con l'ecosistema Kubernetes standard** ✓
- D) `Deployment` è più veloce durante il rollout

5. Quale annotazione Kubernetes popola il campo `CHANGE-CAUSE` visibile in `oc rollout history`?

- A) `openshift.io/change-reason`
- B) `apps.openshift.io/change-cause`
- C) **`kubernetes.io/change-cause`** ✓
- D) `deployment.kubernetes.io/revision-reason`